CoderUJB

Un benchmark Java pour évaluer les LLM

Zeng et al. – ISSTA 2024

Présenté par

Wei WEI Hao QIAN Jianye SHI

Xiang BIAN Zijie Fang Yupan Huang

Master CHPS – UVSQ

Évaluation de performances

Article de Zhengran Zeng, Yidong Wang, Rui Xie, Wei Ye et Shikun Zhang

Sommaire

1. Contexte de la recherche

2. Contributions et méthode

3. Résultats : contexte et modèles

4. Résultats : effets de l'entraînement

5. Limites de l'étude

6. Critique et conclusion

1. Contexte de la recherche

Wei WEI

Comment évaluer une IA qui code ?

Les LLM peuvent aider à programmer

Proposer du code · Corriger une erreur

À LA LECTURE

Un code peut **sembler correct**

Il faut donc le tester.

À L'EXÉCUTION

Pourtant, il peut **échouer**

Pour comparer les modèles : un benchmark

Des tâches et des règles communes

Un exercice isolé a ses limites

HUMANEVAL

164 exercices Python

Compléter une fonction

Des tests vérifient le résultat

Mais les exercices restent
assez isolés.

DANS UN VRAI PROJET

D'autres fichiers

Des bibliothèques

Du code qui existe déjà

Exercice de cours $\neq$ logiciel complet

Réussir le premier ne garantit pas qu'on réussira le second.

Développer, c'est faire plusieurs tâches

Écrire du code **Créer des tests** **Trouver des erreurs** **Les corriger**

Une seule tâche donne une **vision limitée.**

PLUSIEURS TÂCHES, MAIS D'AUTRES LIMITES

Selon les auteurs

CodeXGLUE	Ressemblance entre les textes
XCodeEval	Problèmes de concours

Ces approches ne réunissent pas toutes les conditions recherchées.

Le besoin : une évaluation plus réaliste

Réunir trois éléments

Exécuter les solutions

Vérifier leur fonctionnement

De vrais projets

Code et environnement

Plusieurs types de tâches

Des capacités variées

CoderUJB – un benchmark Java

Comment mieux évaluer les capacités des modèles dans des situations proches du travail réel ?

2. Contributions et méthode

Hao QIAN

Un benchmark commun pour cinq tâches

2 239	17	5
questions	projets Java	tâches

Source commune : Defects4J. Un benchmark et une étude comparative.

Les cinq tâches de CoderUJB

Tâche	**Action demandée**	**Questions**
FCG	Compléter une fonction	238
CTG	Créer un test à partir du code	140
ITG	Reproduire un bug signalé	451
APR	Corriger une fonction	470
DD	Détecter un bug : oui / non	940

Total : 2 239 questions. DD comprend 470 versions défectueuses et 470 versions corrigées.

Du projet à la réponse du modèle

Sélection

- Extraction, filtres et contrôle manuel

Prompt

- Consigne et contexte adaptés à la tâche

Évaluation

- Solutions replacées dans le projet

Projet complet disponible $\neq$ projet complet envoyé dans chaque prompt.

Des critères adaptés à chaque tâche

Génération

- 20 propositions par question ; pass-all@1 et pass-all@10

Tests de bugs

- Échec sur la version défectueuse, succès après correction

Détection

- Accuracy : proportion de classifications correctes

Syntaxe correcte, compilation et réussite des tests sont des étapes différentes.

3. Résultats : contexte et modèles

Jianye SHI

Le contexte du projet aide la génération

```
package org.apache.commons.math3.stat.regression;
import org.apache.commons.math3.linear.LUDecomposition;
public class GLSMultipleLinearRegression extends AbstractMultipleLinearRegression {
    private RealMatrix Omega;
    private RealMatrix OmegaInverse;

    protected RealVector calculateBeta();
    protected RealMatrix calculateBetaVariance();
}
```

Imports

Champs

Signatures

Fig. 2(a), extrait adapté : annotations traduites ; ce n'est pas le prompt complet.

CodeLlama-13B

Génération de fonctions (FCG)

Trois prompts, un même modèle.

Prompt	**pass-all@1**
Un exemple	12,84 %
Quatre exemples	14,50 %
Contexte du projet	21,91 %

Source : Zeng et al., ISSTA 2024, fig. 2(a) et tableau 3.

Un écart plus marqué en réparation

FCG : générer une fonction

APR : réparer une fonction

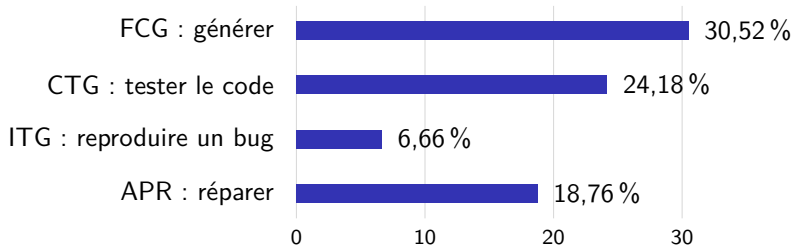
Modèle	**FCG**	**APR**
CodeLlama-34B	22,82 %	5,01 %
GPT-3.5-Turbo	23,37 %	9,31 %
GPT-4	**30,52 %**	**18,76 %**

Écart GPT-4 / CodeLlama : **7,70 points** en FCG, **13,75** en APR.

pass-all@1. Source : Zeng et al., tableau 4.
Versions : GPT-3.5-Turbo-0301 et GPT-4-0314.

GPT-4 : des scores différents selon la tâche

GPT-4-0314



Réussite : FCG / APR : passer les tests.
CTG : test exécutable couvrant le code.
ITG : test en échec avec le bug, puis réussi après correction.

Source : Zeng et al., §3.5 et tableau 4.

Détecter les bugs reste difficile

940	50 %	52,16 %
fonctions	réponse constante	GPT-4

470 fonctions avec un bug et 470 versions corrigées.

Accuracy : proportion de réponses correctes.
GPT-4 dépasse la référence de **2,16 points**.

À retenir

Le contexte aide la génération. Les autres tâches restent à vérifier séparément.

Source : Zeng et al., §3.3 et tableau 4 ; GPT-4-0314.

4. Résultats : effets de l'entraînement

Xiang BIAN

Plus d'entraînement = de meilleures performances ?

RQ3 : préentraînement spécialisé par langage

- Continuer l'entraînement avec du code Python ou Java

RQ4 : ajustement sur instructions

- Apprendre à suivre des demandes grâce à des exemples de réponses

Question commune : quel effet sur les tâches réelles de CoderUJB ?

Se spécialiser dans un langage : un compromis

StarCoder-15B

CoderUJB FCG / Java

Base	15,32 %
+ Python	14,39 %
+ Java	18,82 %

HumanEval / Python

Base	30,35 %
+ Python	33,57 %
+ Java	27,07 %

En FCG, la spécialisation favorise généralement le langage ciblé.

Mais ce résultat n'est pas systématique pour ITG et APR (RQ3).

L'instruction tuning n'améliore pas toutes les tâches

CodeLlama-13B

Tâche et indicateur	Base	Instruct
FCG : pass-all@1	21,91 %	13,28 %
CTG : pass-all@1	12,61 %	6,14 %
APR : pass-all@1	4,50 %	4,09 %
APR : pass-all@10	8,39 %	15,72 %

Tableau 5 : pourquoi la génération se dégrade-t-elle ?
FCG, CodeLlama-13B

Indicateur	Base	Instruct
Syntaxe	94,14 %	95,99 %
Compilation	63,49 %	40,76 %

La syntaxe reste correcte, mais le code compile moins souvent.

Plus d'entraînement $\neq$ meilleures performances

RQ3 : préentraînement par langage

La spécialisation aide certaines tâches, mais ses effets varient selon le langage et la tâche.

RQ4 : instruction fine-tuning

La génération peut se dégrader ; certaines réparations peuvent s'améliorer.

Interprétation des auteurs

La proximité entre préentraînement et tâche évaluée pourrait expliquer ces différences.

Cas particulier : CodeLlama-Instruct-34B produit des réponses répétées (mode collapse).

5. Limites de l'étude

Zijie Fang

Les trois limites principales

Validité interne

- Erreurs possibles dans les scripts

Validité externe

- Choix des modèles et généralisation

Validité des mesures

- Indicateurs et qualité des tests

Les auteurs cherchent à limiter ces risques.

Réussir les tests ne suffit pas

Ce que font les auteurs

- Vérification des scripts et publication des résultats

Limite des tests

- Un programme peut encore contenir des bugs

Limite de la couverture

- Exécuter du code $\neq$ vérifier tous ses comportements

Passer les tests $\neq$ correction garantie.

Des résultats difficiles à généraliser

Modèles

- Des versions précises des LLM

Données

- 17 projets Java issus de Defects4J

Réparation (APR)

- Des bugs dans une seule fonction

Pas être généralisés à tous les cas.

Une évaluation encore limitée

Protocole étudié

- Prompt → réponse → évaluation

Autre approche possible

- Tester → corriger → recommencer

Contexte fourni

- Le modèle ne reçoit pas forcément tout le projet

Ces limites précisent la portée des conclusions.

6. Critique et conclusion

Yupan Huang

Notre évaluation : un apport utile

Pertinence

- Plusieurs activités réelles de développement

Vérification

- Exécution et critères adaptés aux tâches

Transparence

- Code, scripts et résultats rendus publics

Notre appréciation : une contribution à la méthode d'évaluation, au-delà d'un classement.

Des comparaisons à mieux isoler

Constat du protocole

- Base : complétion ; Instruct : dialogue

Réserve

- Le format peut contribuer à l'écart observé

Amélioration proposée

- Tester plusieurs formats et quantifier l'incertitude

Un contenu équivalent ne garantit pas un effet identique du format.

Une lecture attentive des chiffres

GPT-4, tâche ITG

Indicateur	**Valeur du tableau 4**
pass-all@1	6,66 %
pass-all@10	15,76 %

RQ2 mentionne 15,76 parmi des scores annoncés à $k = 1$. Nos diapositives suivent le tableau 4.

Conclusion et pistes de prolongement

Élargir

- Autres projets, langages et méthodes avec outils

Contrôler

- Risque de contamination et sensibilité au protocole

Retenir

- Comparer par tâche, avec la même métrique

CoderUJB : une base utile pour une évaluation plus réaliste, dans un périmètre défini.